

PRÁCTICA FASE DE DISEÑO

Videojuego de Combate

Autores:

Lucca Manfredotti García
Sergio Alejandro Romero López
Neville Gil Ortiz

Año académico: 2do

Asignatura: Metodología de la Programación

Ingeniería del Software

Tabla de contenido

1. Diagramas de clases	3
A. Diagrama de clases 1.....	3
Patrón State.....	3
B. Diagrama de clases 2.....	4
Patrón Strategy	4
C. Diagrama de clases 3.....	5
Patrón Abstract Factory	5
D. Diagrama de clases 4.....	6
Patrón Observer	6
E. Diagrama de clases 5.....	7
2. Diagramas de actividad	8
A. Diagrama de actividad 1.....	8
B. Diagrama de actividad 2.....	9
3. Diagramas de secuencia	11
A. Diagrama de secuencia 1.....	11
B. Diagrama de secuencia del inicio de sesión	12
C. Diagrama de secuencia Edición Personaje y Añadir Equipo	13
D. Diagrama de secuencia Rechazar Desafío	14
4. Diagramas de estados	15
A. Diagrama de estado del proceso del desafío	15
B. Diagramas de estados del combate	16
5. Diagramas de casos de uso	17
Anexo	19
Datos de ejemplo	19
Entidad de Ejemplo 1.....	19
Entidad de ejemplo 2	20
Prototipos de pantallas	21

Tabla de ilustraciones

Ilustración 1 Diagrama de clases 1	3
Ilustración 2 Diagrama de clases 2	4
Ilustración 3 Diagrama de actividad 1	8
Ilustración 4 Diagrama de actividad 2	9
Ilustración 5 Diagrama de secuencia 1	11
Ilustración 6 Diagrama de secuencia 2	12
Ilustración 7 Diagrama de estados 1	15
Ilustración 8 Diagrama de estados 2	16
Ilustración 9 - Diagrama de casos de uso 1	17
Ilustración 10 - Diagrama de caso de uso 2	18
Ilustración 11 - Entidad de Ejemplo 1	19
Ilustración 12 - Entidad ejemplo 2	20
Ilustración 13 Pantalla de bienvenida	21
Ilustración 14 Pantalla de inicio de sesión	21

1. Diagramas de clases

Los Diagramas de Clases muestran la estructura estática del sistema, es decir, las entidades que lo componen, sus atributos, métodos y las relaciones entre ellas. Se han dividido en dos diagramas complementarios porque el sistema tiene suficiente complejidad como para que un único diagrama resultara ilegible.

A. Diagrama de clases 1

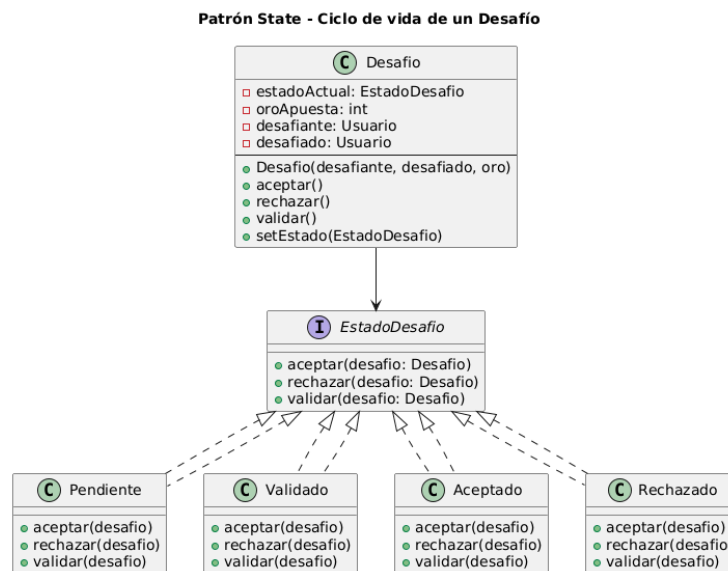


Ilustración 1 Diagrama de clases 1

Patrón State

- ✓ **Problema que resuelve:** Un desafío pasa por diferentes estados (Pendiente, Validado, Aceptado, Rechazado) y cada estado tiene comportamientos diferentes. Sin este patrón, tendríamos una clase llena de condicionales if-else para verificar el estado antes de cada acción.
- ✓ **Ventajas:** Cada estado encapsula su propia lógica. Añadir un nuevo estado (ej: "EnProceso") solo requiere crear una nueva clase sin modificar las existentes. El código es más limpio y mantenible.
- ✓ **Desafío (Contexto):** Es la clase principal que gestiona un desafío entre jugadores. Mantiene una referencia al estado actual y delega en él todas las operaciones. Contiene información como el oro apostado, quién desafía y quién es desafiado. El jugador interactúa con esta clase, pero el comportamiento real depende del estado en el que se encuentre.
- ✓ **EstadoDesafio (Interfaz):** Define el contrato que deben cumplir todos los estados. Declara los métodos aceptar (), rechazar () y validar () que cada estado implementará de forma diferente. Esta interfaz permite que el Desafío interactúe con cualquier estado sin conocer su implementación concreta.
- ✓ **Pendiente (Estado Concreto):** Representa el estado inicial cuando un desafío es creado. En este estado, solo es posible validar el desafío por parte del operador. Si se intenta aceptar o rechazar directamente, el comportamiento no está permitido o lanza una excepción.

B. Diagrama de clases 2

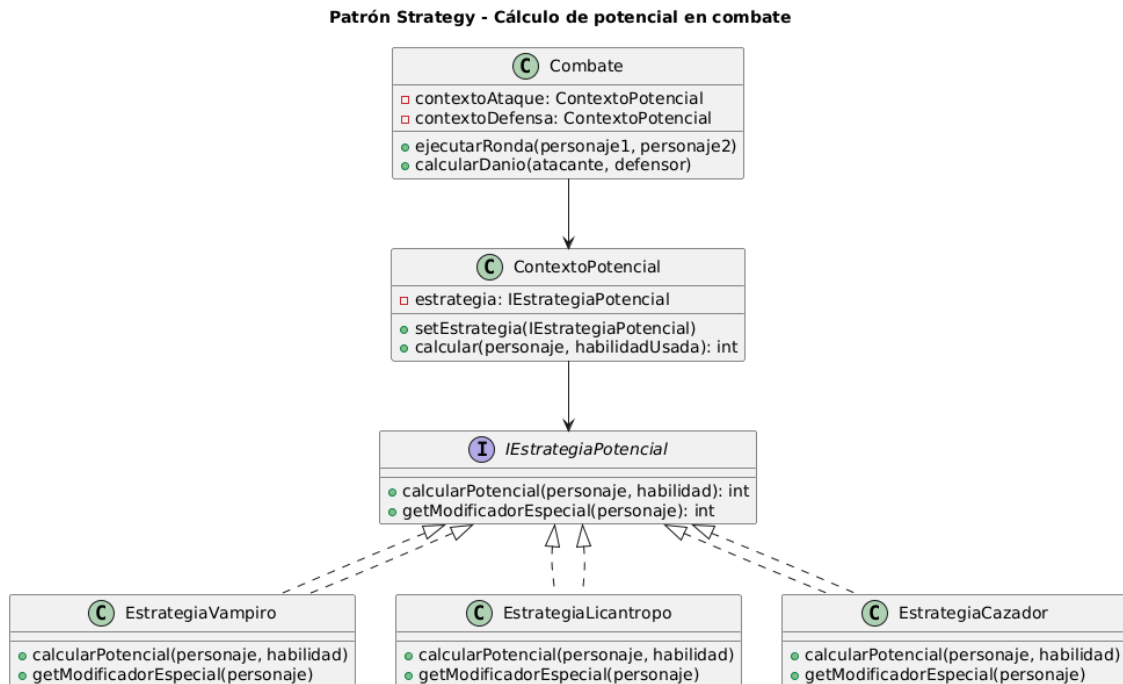
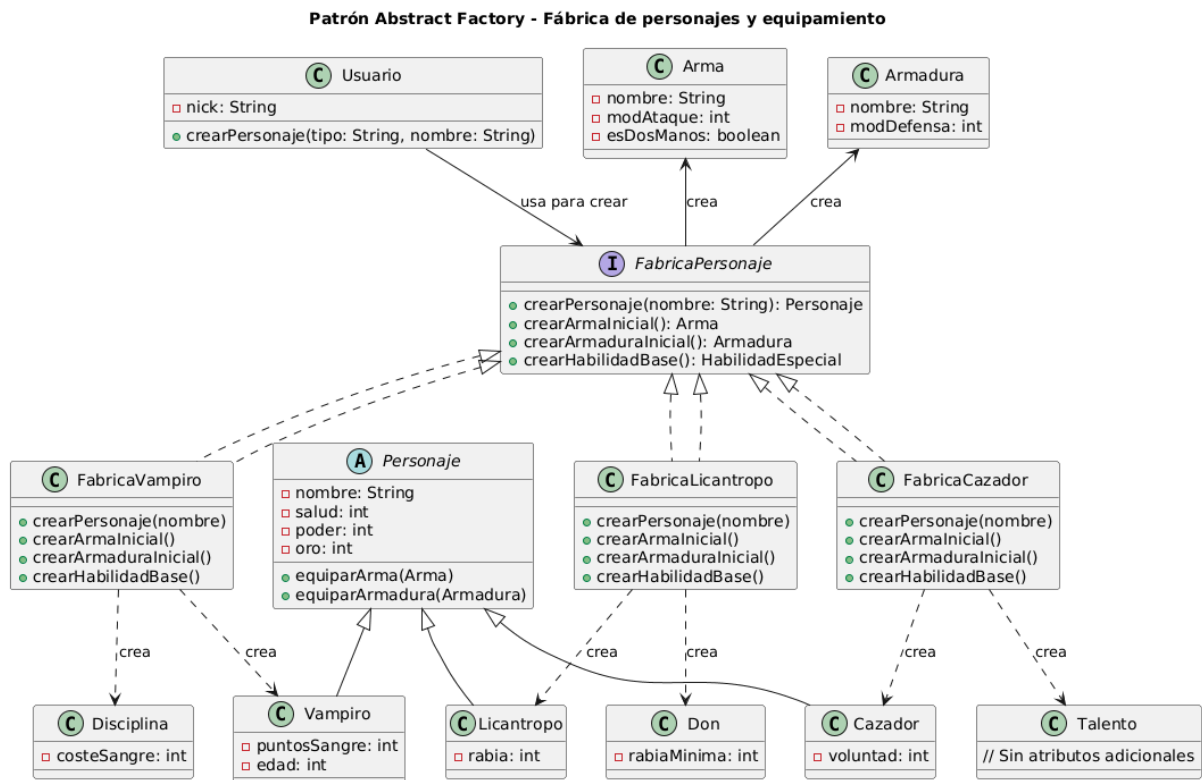


Ilustración 2 Diagrama de clases 2

Patrón Strategy

- ✓ **Problema que resuelve:** Vampiros, Licántropos y Cazadores calculan su potencial de ataque/defensa de forma completamente diferente. Sin Strategy, la clase Combate tendría que conocer todos los tipos de personajes y usar condicionales.
- ✓ **Ventajas:** Separa los algoritmos en clases independientes. Añadir un nuevo tipo de personaje (ej: "Elfo") solo requiere crear una nueva estrategia. Promueve el principio Open/Closed (abierto para extensión, cerrado para modificación).
- ✓ **ContextoPotencial:** Es la clase que utiliza el algoritmo. Contiene una referencia a una estrategia concreta y un método calcular () que delega en esa estrategia. El combate usa esta clase tanto para ataque como para defensa, pudiendo cambiar la estrategia según el personaje que esté actuando.
- ✓ **IEstrategiaPotencial (Interfaz):** Define el método calcularPotencial() que todas las estrategias deben implementar. Esta interfaz permite que el ContextoPotencial trabaje con cualquier tipo de personaje sin conocer los detalles específicos de cómo calculan su potencial.
- ✓ **EstrategiaVampiro:** Implementa el cálculo específico para vampiros. Su fórmula es: poder + valor de disciplina + equipo + (2 si puntos de sangre >= 5). Además, descuenta el coste en sangre al usar la disciplina y recupera 4 puntos si el ataque tiene éxito.
- ✓ **EstrategiaLicantropo:** Implementa el cálculo para licántropos. Su fórmula es: poder + valor del don + equipo + rabia actual. También verifica que la rabia del personaje sea suficiente para usar el don seleccionado.
- ✓ **EstrategiaCazador:** Implementa el cálculo más simple para cazadores. Su fórmula es: poder + valor del talento + equipo + voluntad actual. La voluntad disminuye cuando el cazador pierde salud.

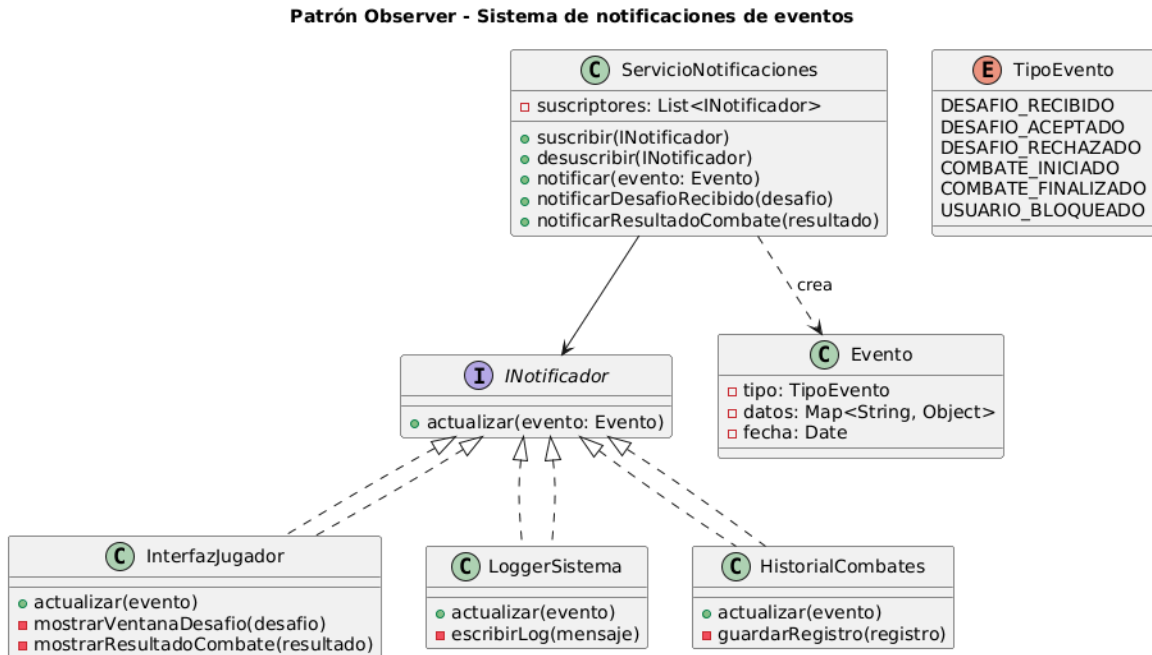
C. Diagrama de clases 3



Patrón Abstract Factory

- ✓ **Problema que resuelve:** Crear un personaje implica crear también su equipamiento inicial y habilidades base, que están relacionados entre sí. Un vampiro no puede tener habilidades de cazador. Sin este patrón, el código cliente tendría que conocer todas las clases concretas.
- ✓ **Ventajas:** Asegura que los objetos creados sean compatibles entre sí. El Usuario solo necesita saber qué tipo de personaje quiere, no cómo se construye. Facilita añadir nuevas razas de personajes.
- ✓ **FabricaPersonaje (Interfaz):** Define los métodos para crear una familia completa de objetos relacionados: el personaje, su arma inicial, su armadura inicial y su habilidad base. Esta interfaz asegura que todos los objetos creados por una misma fábrica sean compatibles entre sí.
- ✓ **FabricaVampiro (Fábrica Concreta):** Implementa la creación de objetos específicos para vampiros. Crea un objeto Vampiro, le asigna un arma inicial apropiada, una armadura inicial y una disciplina como habilidad base. Todos los objetos devueltos son coherentes con la temática vampírica.
- ✓ **FabricaLicantropo:** Similar a la anterior, pero para licántropos. Crea un Licantropo con su equipamiento característico y un Don como habilidad especial.
- ✓ **FabricaCazador:** Crea personajes de tipo cazador con sus talentos correspondientes. Aunque los cazadores no tienen mecánicas adicionales como la sangre o la rabia, siguen necesitando equipamiento y habilidades.
- ✓ **Usuario:** Es el cliente que utiliza la fábrica. Cuando un jugador quiere crear un personaje de un tipo específico, el Usuario recibe la fábrica correspondiente (por ejemplo, **FabricaVampiro**) y la usa para crear todos los objetos necesarios sin preocuparse por cómo se construyen internamente.

D. Diagrama de clases 4

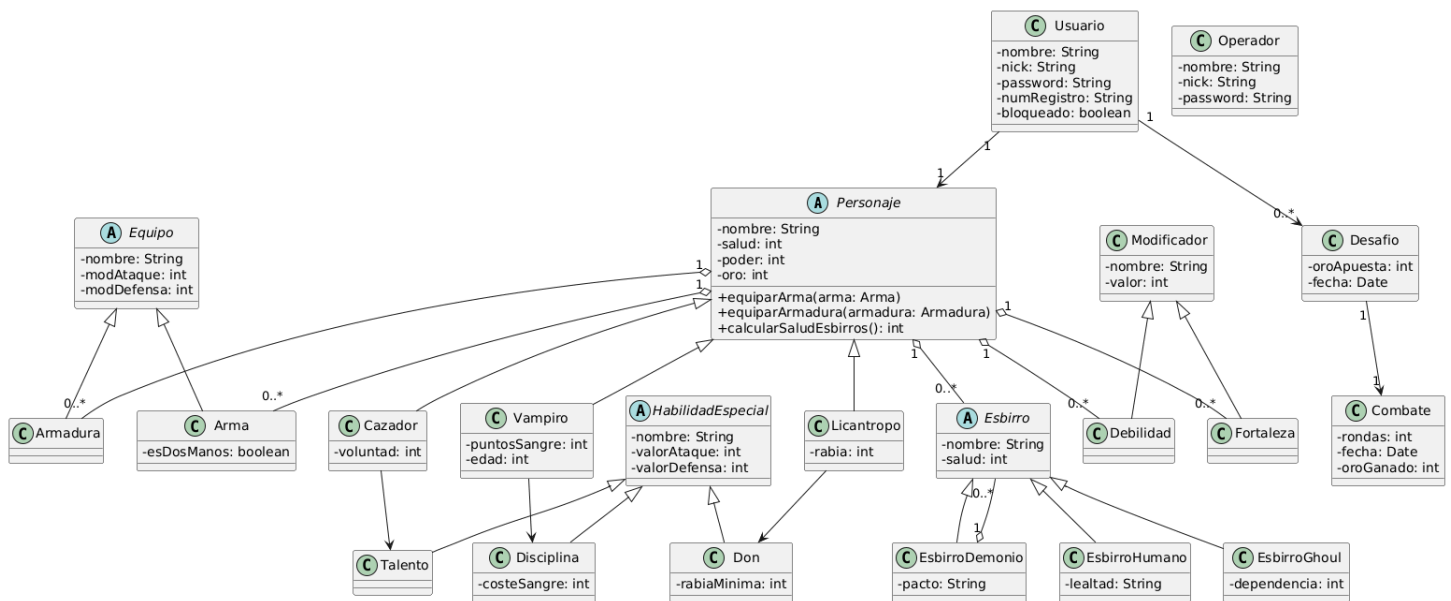


Patrón Observer

- ✓ **Problema que resuelve:** Cuando ocurre un evento importante (desafío recibido, combate finalizado), múltiples componentes deben reaccionar: mostrar notificación al jugador, registrar en log, guardar en historial. Sin Observer, el emisor del evento tendría que conocer todos estos componentes.
- ✓ **Ventajas:** Desacopla el emisor de los receptores. Se pueden añadir nuevos observadores (ej: "Sistema de Trofeos", "Analítica") sin modificar el código del ServicioNotificaciones. Soporta relaciones uno-a-muchos de forma limpia.
- ✓ **ServicioNotificaciones (Sujeto):** Es el centro de notificaciones del sistema. Mantiene una lista de todos los suscriptores interesados en recibir eventos. Cuando ocurre algo importante (un desafío recibido, un combate finalizado), recorre la lista y llama al método actualizar () de cada suscriptor, pasándoles la información del evento.
- ✓ **Notificador (Interfaz):** Define el método actualizar () que todos los observadores deben implementar. Esta interfaz permite que el ServicioNotificaciones notifique a cualquier tipo de observador sin conocer su implementación concreta.
- ✓ **InterfazJugador (Observador Concreto):** Es el responsable de mostrar información al jugador en tiempo real. Cuando recibe una notificación de desafío, muestra una ventana emergente. Cuando recibe un resultado de combate, presenta la pantalla de resultados con las rondas y el oro ganado/perdido.
- ✓ **LoggerSistema (Observador Concreto):** Registra todos los eventos en archivos de log para auditoría y depuración. Es un observador "silencioso" que no interfiere con la experiencia del usuario, pero mantiene un registro histórico de todas las acciones importantes.
- ✓ **HistorialCombates (Observador Concreto):** Se encarga de la persistencia. Cuando un combate finaliza, guarda toda la información relevante (fecha, rondas, vencedor, oro, esbirros supervivientes) en la base de datos o sistema de archivos para que el jugador pueda consultar su historial más tarde.

E. Diagrama de clases 5

Este diagrama muestra la estructura completa del sistema, representando todas las entidades principales y sus relaciones. Sirve como visión global del modelo de datos sobre el que se aplican los patrones de diseño detallados en los diagramas anteriores. Se distinguen tres jerarquías principales: la jerarquía de Personaje (Vampiro, Licántropo y Cazador), la jerarquía de Esbirro (Humano, Ghoul y Demonio) y la jerarquía de HabilidadEspecial (Disciplina, Don y Talento). Además, se representan las clases Usuario, Operador, Equipo, Modificador, Desafío y Combate con sus relaciones.



2. Diagramas de actividad

Los diagramas de actividad describen el flujo de las operaciones principales del sistema mediante acciones, decisiones y transiciones. Se han elaborado dos diagramas: uno para el registro, inicio de sesión y gestión de personaje del jugador, y otro para el proceso completo del desafío y combate, incluyendo los carriles de cada actor implicado.

A. Diagrama de actividad 1

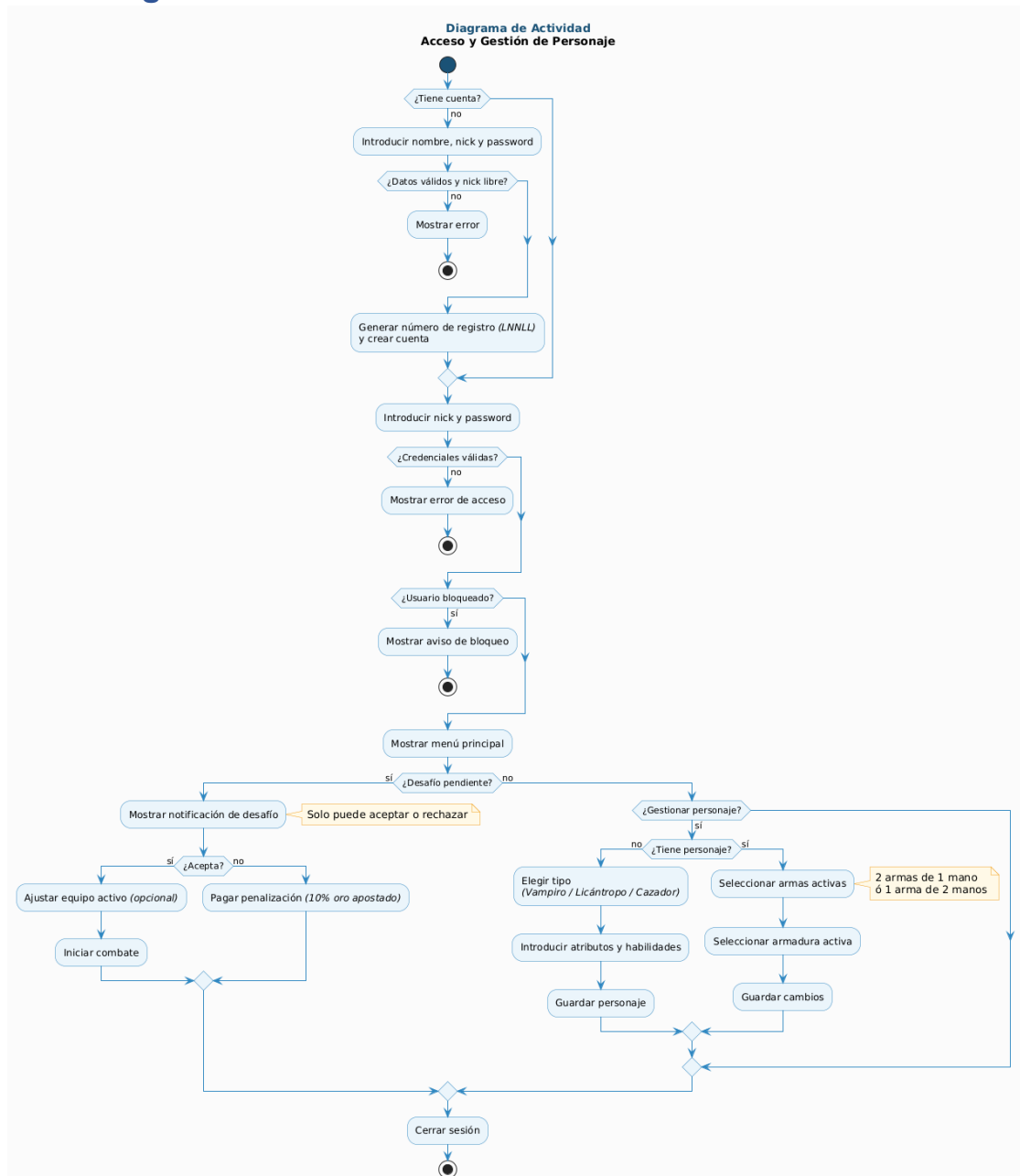


Ilustración 3 Diagrama de actividad 1

Este diagrama muestra el flujo desde que el usuario accede a la aplicación hasta que cierra sesión. Se contemplan dos ramas de entrada: registro de nuevo jugador (con validación de password 8-12 caracteres y nick único) e inicio de sesión (con comprobación de bloqueo). Una vez autenticado, el jugador puede crear o gestionar su personaje, seleccionar el equipo activo, ver el ranking o consultar su historial de oro.

B. Diagrama de actividad 2

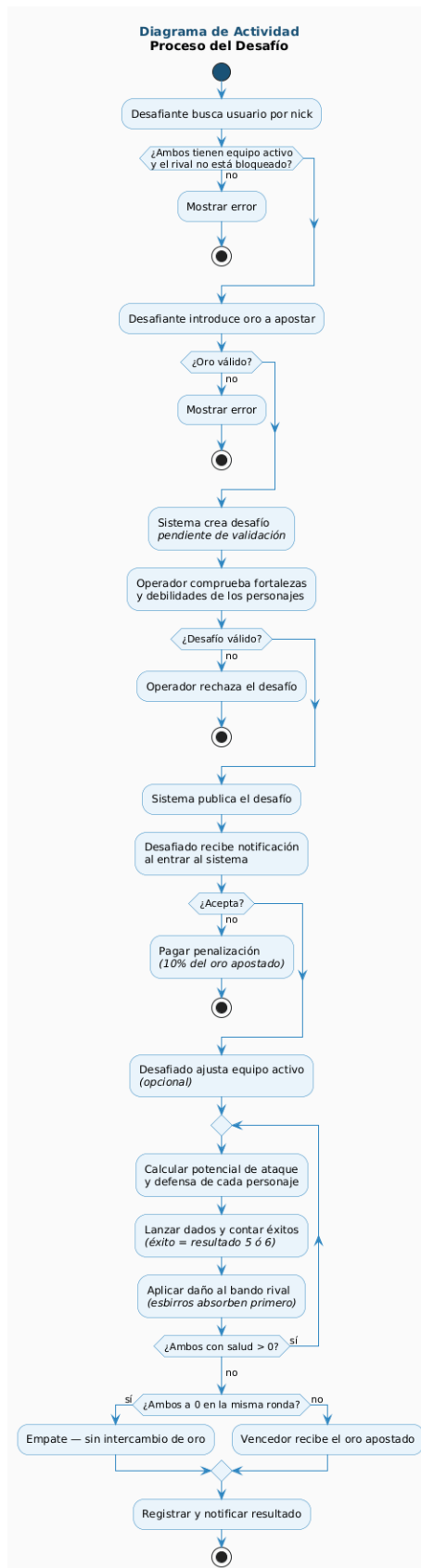


Ilustración 4 Diagrama de actividad 2

Este diagrama representa el ciclo de vida completo de un desafío de forma secuencial. El desafiante busca rival y apuesta oro, el operador valida el desafío revisando fortalezas y debilidades, el desafiado acepta o rechaza pagando penalización del 10% si rechaza, y finalmente el sistema ejecuta las rondas de combate, determina el vencedor y transfiere el oro apostado.

3. Diagramas de secuencia

Los diagramas de secuencia muestran la interacción temporal entre los actores y las capas del sistema. Se detallan dos flujos críticos: el proceso de registro e inicio de sesión, y el proceso completo de desafío y combate, desde la creación hasta la notificación del resultado a ambos jugadores.

A. Diagrama de secuencia 1

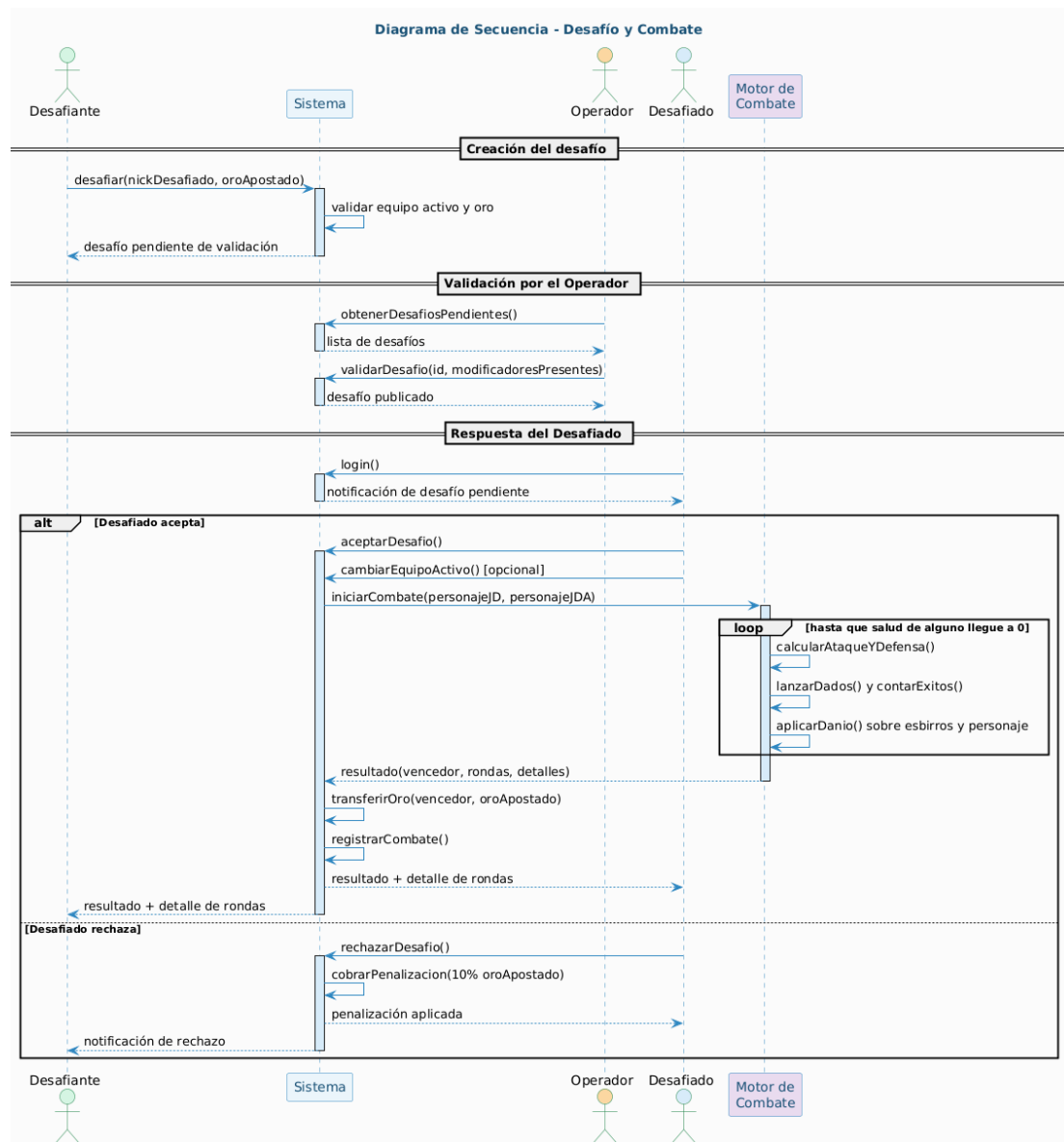


Ilustración 5 Diagrama de secuencia 1

Este diagrama recoge la interacción entre el Jugador Desafiante, el Sistema, el Operador, el Jugador Desafiado y el Motor de Combate. El desafiante crea el desafío; el operador lo valida estableciendo los modificadores presentes; el desafiado responde aceptando o rechazando. Si acepta, el Motor de Combate ejecuta el bucle de rondas (calcularPotencial, lanzarDados, aplicarDaño) hasta que un personaje llega a salud 0, transfiere el oro y registra el resultado.

B. Diagrama de secuencia del inicio de sesión

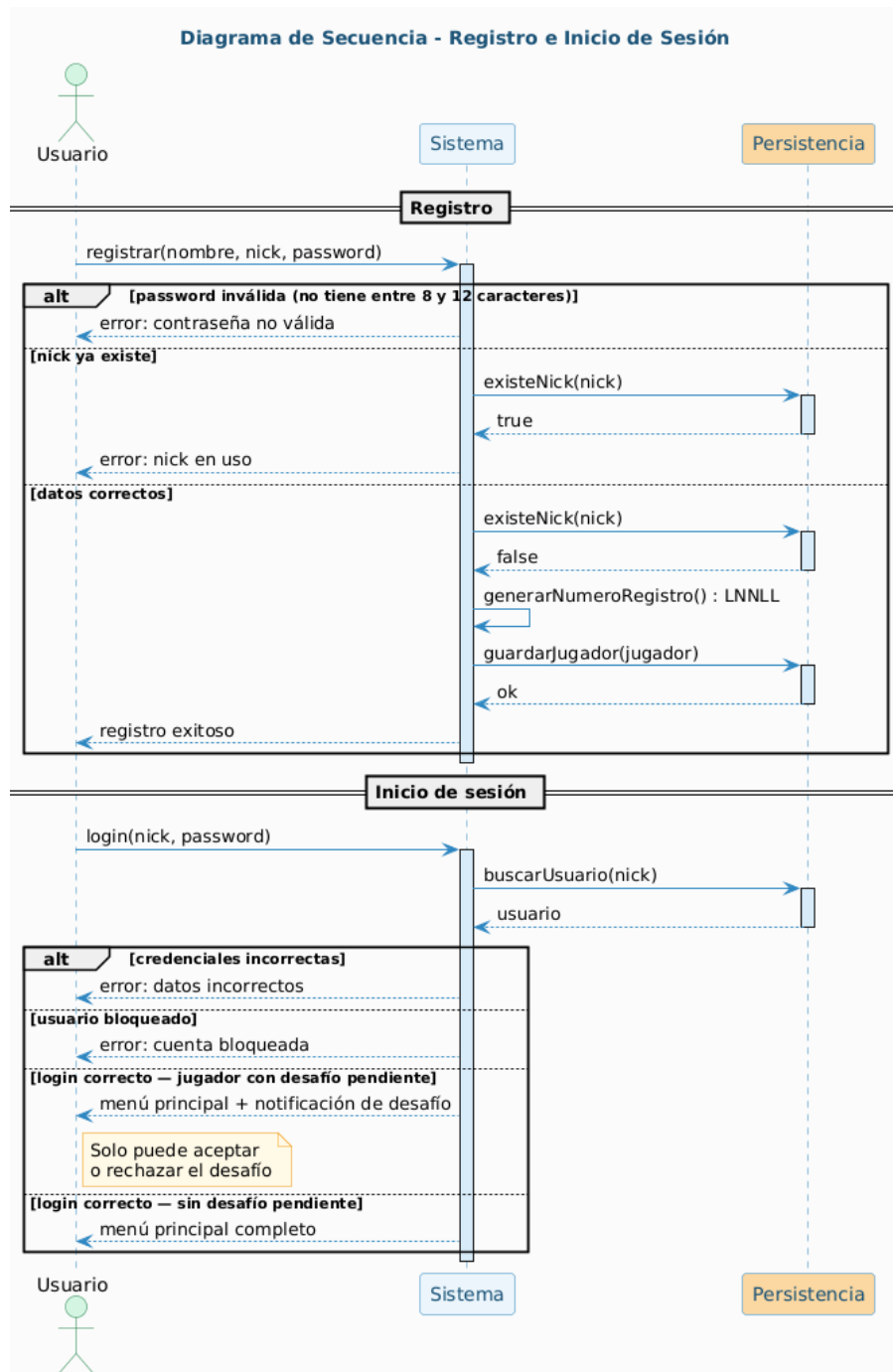
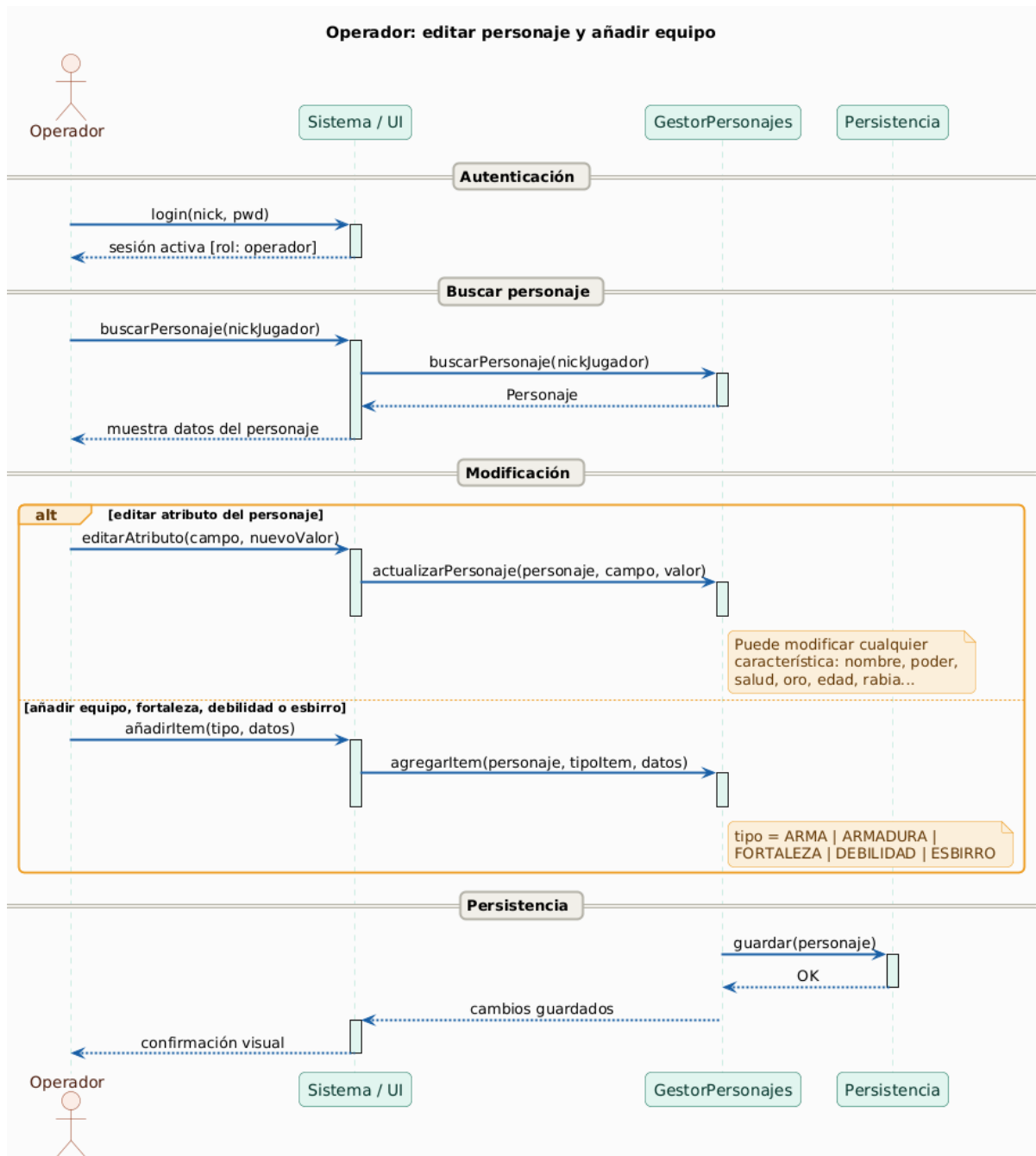


Ilustración 6 Diagrama de secuencia 2

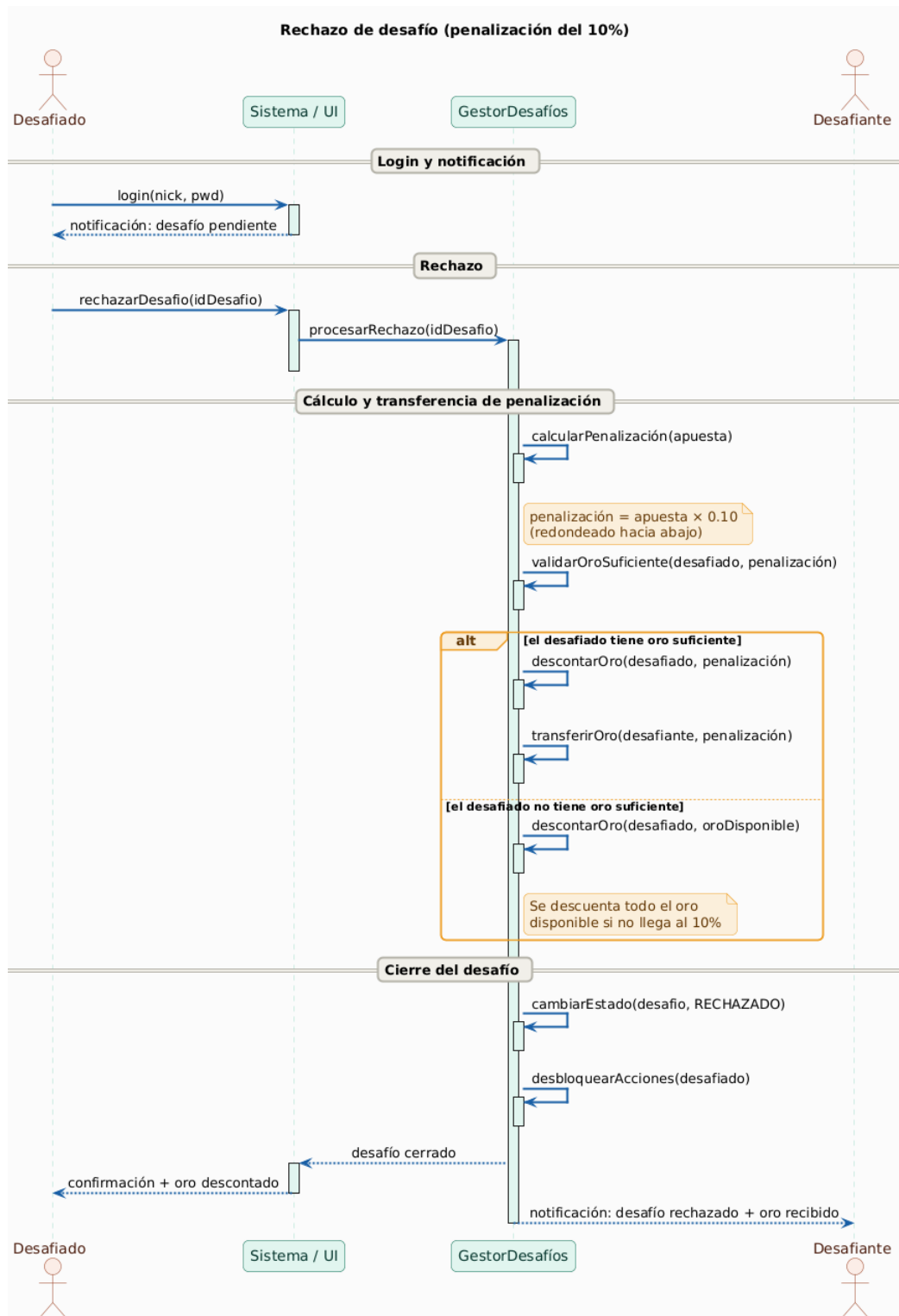
Este diagrama muestra la secuencia de mensajes entre el Usuario, la Interfaz de Usuario, el Controlador de Autenticación, el Repositorio de Usuarios y la Persistencia. En el flujo de registro se valida la contraseña (8-12 caracteres), se comprueba la unicidad del nick y se genera el número de registro LNNLL. En el inicio de sesión se verifican las credenciales, se detecta si el usuario está bloqueado y se muestra el menú adecuado según el rol y los desafíos pendientes.

C. Diagrama de secuencia Edición Personaje y Añadir Equipo



Este diagrama muestra la secuencia de mensajes entre el Operador, la Interfaz de Usuario, el GestorPersonajes y la Persistencia. En el flujo de autenticación se verifica que el usuario tiene rol de operador y se inicia la sesión. A continuación, se busca el personaje a modificar por el nick del jugador propietario. En la fase de modificación se distinguen dos alternativas: editar un atributo concreto del personaje (nombre, poder, salud, oro, edad, rabia, voluntad o puntos de sangre) o añadir un nuevo elemento de tipo arma, armadura, fortaleza, debilidad o esbirro.

D. Diagrama de secuencia Rechazar Desafío



Este diagrama muestra la secuencia de mensajes entre el Desafiado, la Interfaz de Usuario, el GestorDesafíos y el Desafiante. Cuando el desafiado inicia sesión, el sistema le notifica el desafío pendiente y bloquea cualquier otra acción hasta que lo resuelva.

4. Diagramas de estados

Un Desafío comienza en el estado PendienteValidación en cuanto el jugador lo crea con equipo activo y oro suficiente. El operador lo revisa: si es válido pasa a Publicado (visible para el desafiado); si no, queda en Inválido. Desde Publicado, el desafiado puede Rechazarlo (penalización automática del 10%) o Aceptarlo, pasando a EnCombate. Tras resolverse el combate, el desafío alcanza el estado Completado, con el oro transferido al vencedor.

A. Diagrama de estado del proceso del desafío

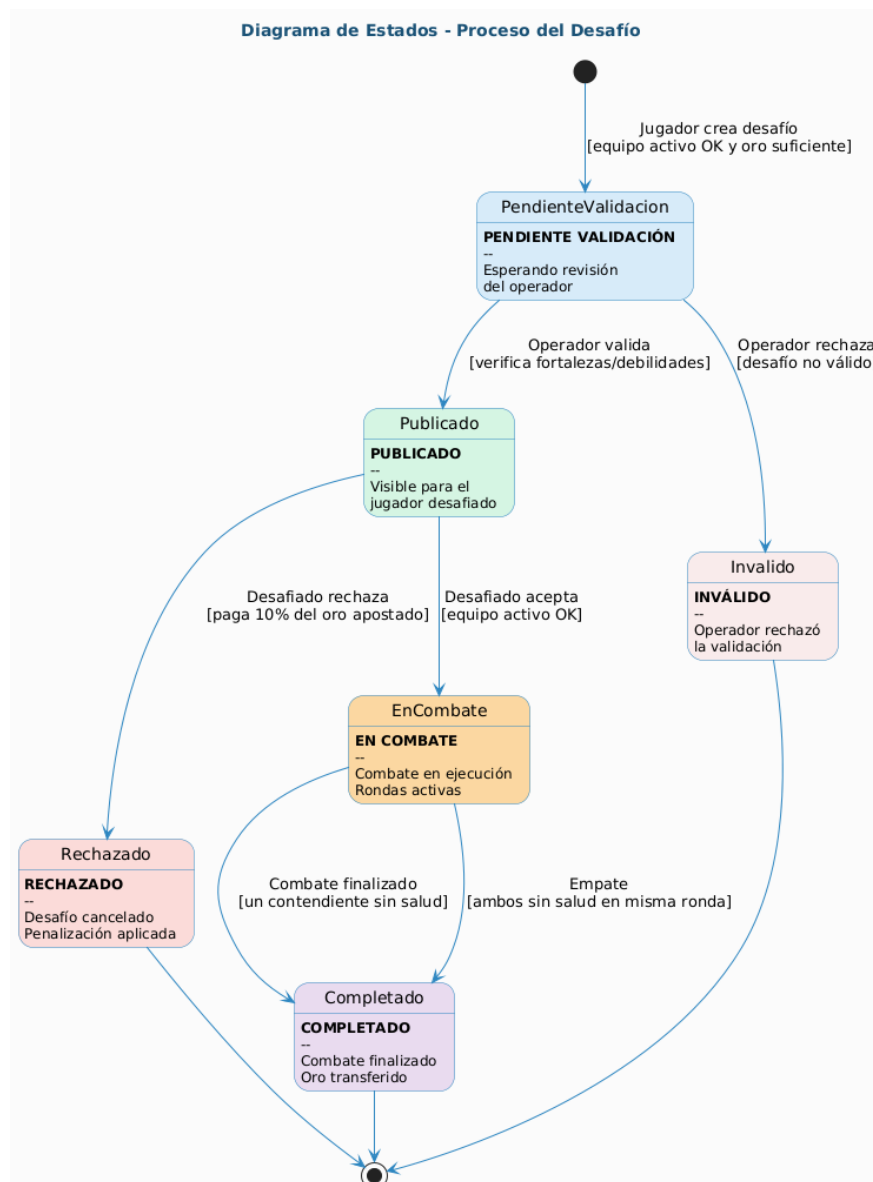


Ilustración 7 Diagrama de estados 1

El diagrama de estados del desafío está directamente relacionado con el patrón State definido en el diagrama de clases 1, donde cada estado (PendienteValidacion, Publicado, EnCombate, Completado, Rechazado e Invalido) se corresponde con una clase concreta que implementa la interfaz EstadoDesafio.

B. Diagramas de estados del combate

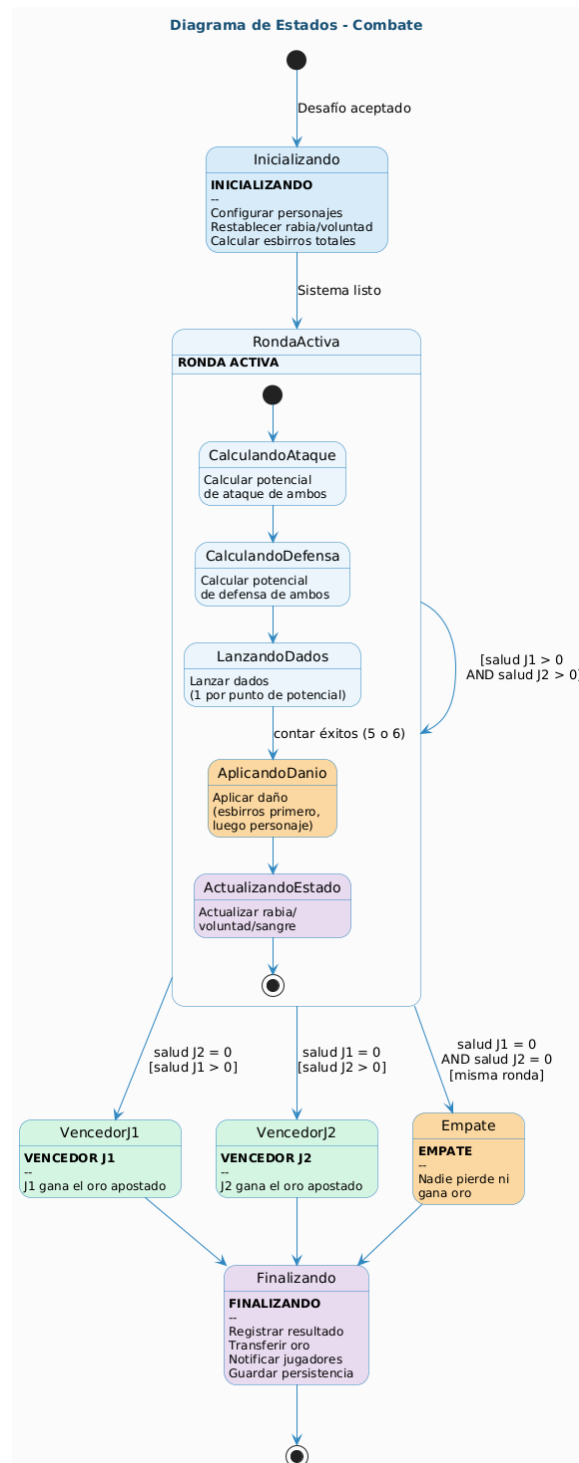


Ilustración 8 Diagrama de estados 2

El combate se inicia en el estado Inicializando, donde se configuran los personajes y se restablecen rabia/voluntad. A continuación, entra en el subestado RondaActiva, que itera por CalculandoAtaque, CalculandoDefensa, LanzandoDados, AplicandoDaño y ActualizandoEstado hasta que la salud de alguno llega a 0. Según el resultado se transita a VencedorJ1, VencedorJ2 o Empate. Finalmente, el estado Finalizando registra el resultado, transfiere el oro y guarda la persistencia.

5. Diagramas de casos de uso

Los diagramas de casos de uso identifican los actores del sistema y las funcionalidades que cada uno puede realizar. Se han definido dos actores principales: el Jugador (usuario registrado con personaje) y el Operador (administrador del sistema). Estos diagramas se mantienen consistentes con los definidos en la fase de análisis y se incluyen aquí para garantizar la autocontenibilidad del documento de diseño.

Diagrama de casos de uso 1 — Jugador

Este diagrama representa las acciones que puede realizar el Jugador dentro del sistema desde el punto de vista del diseño. El registro incluye la generación automática del número LNNLL como proceso del sistema. La gestión del personaje permite crear, dar de baja y equipar armas y armaduras activas respetando las restricciones de una o dos manos. Los desafíos siguen el flujo definido por el patrón State: el jugador solo puede aceptar o rechazar cuando el desafío está en estado Publicado. La consulta del ranking y el historial de oro son operaciones de solo lectura sobre la persistencia.



Ilustración 9 - Diagrama de casos de uso 1

Diagrama de casos de uso 2 — Operador

Este diagrama representa las acciones del Operador desde el punto de vista del diseño. La validación de desafíos es la acción central del operador e incluye obligatoriamente la gestión de modificadores activos mediante relaciones «include», activando o desactivando fortalezas y debilidades de cada contendiente antes de publicar el desafío. La edición de personajes incluye añadir armas, armaduras, esbirros, fortalezas y debilidades. El bloqueo y desbloqueo de jugadores gestiona el acceso al sistema.

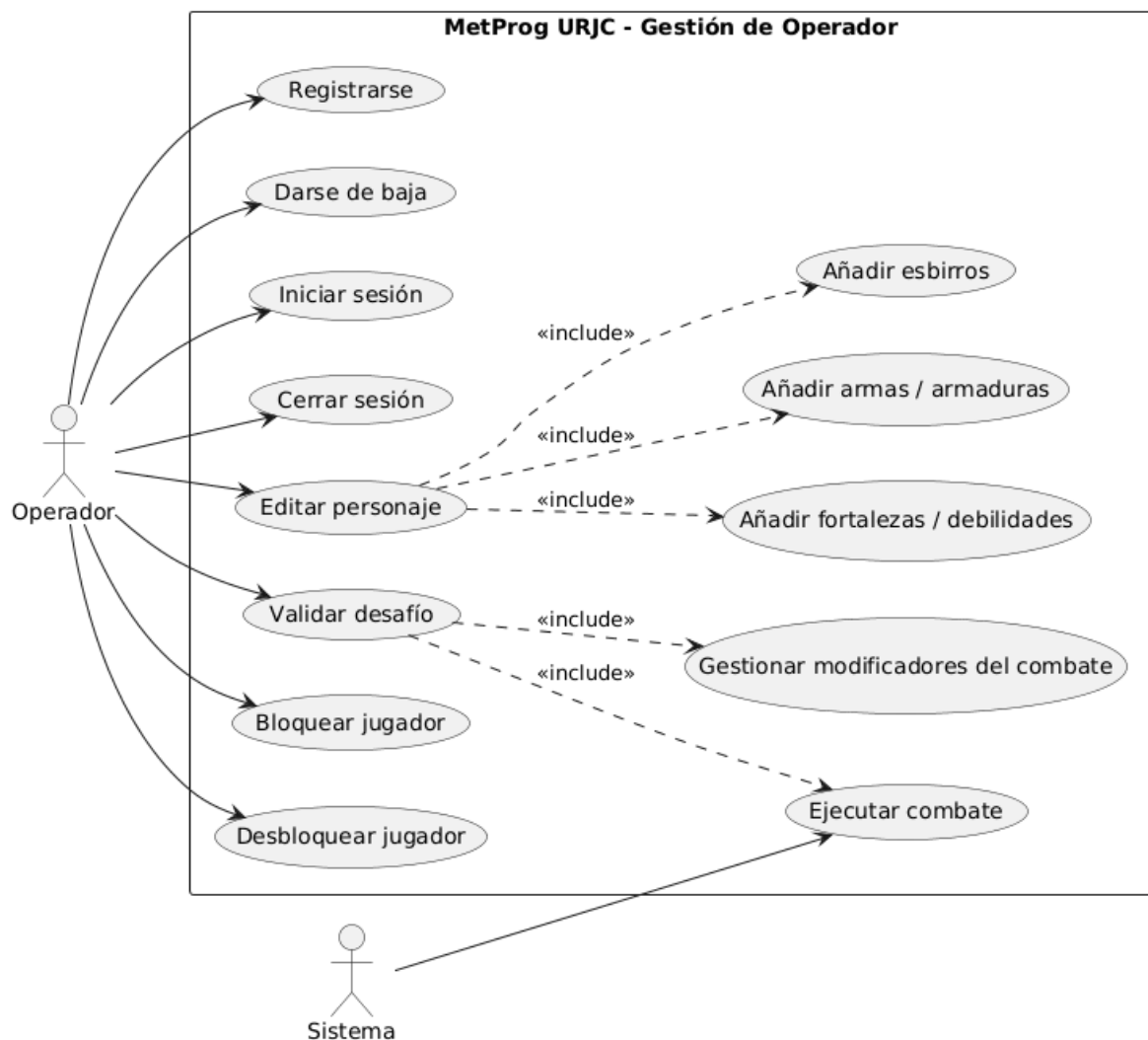


Ilustración 10 - Diagrama de caso de uso 2

Anexo

Datos de ejemplo

Entidad de Ejemplo 1

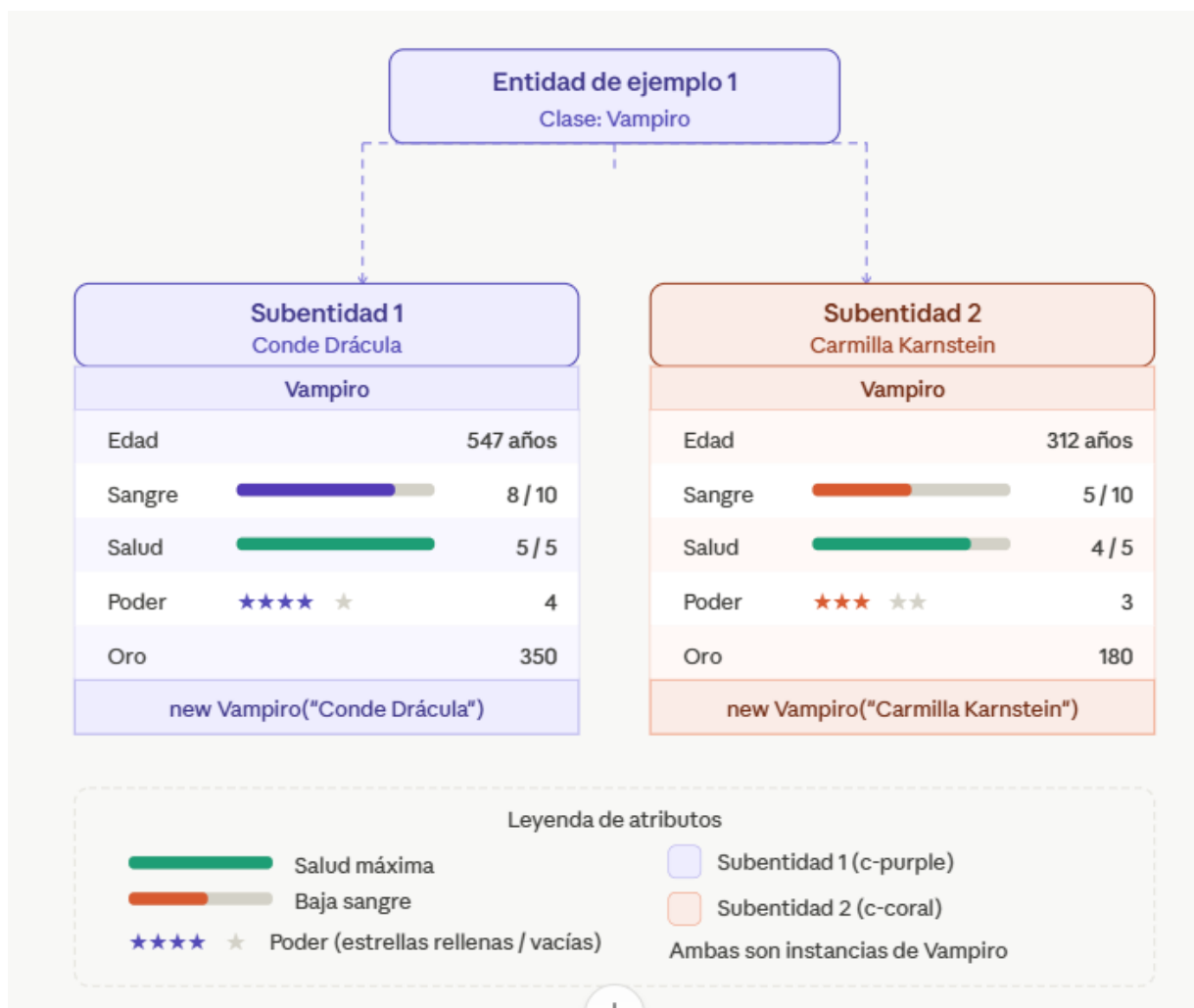


Ilustración 11 - Entidad de Ejemplo 1

Entidad de ejemplo 2

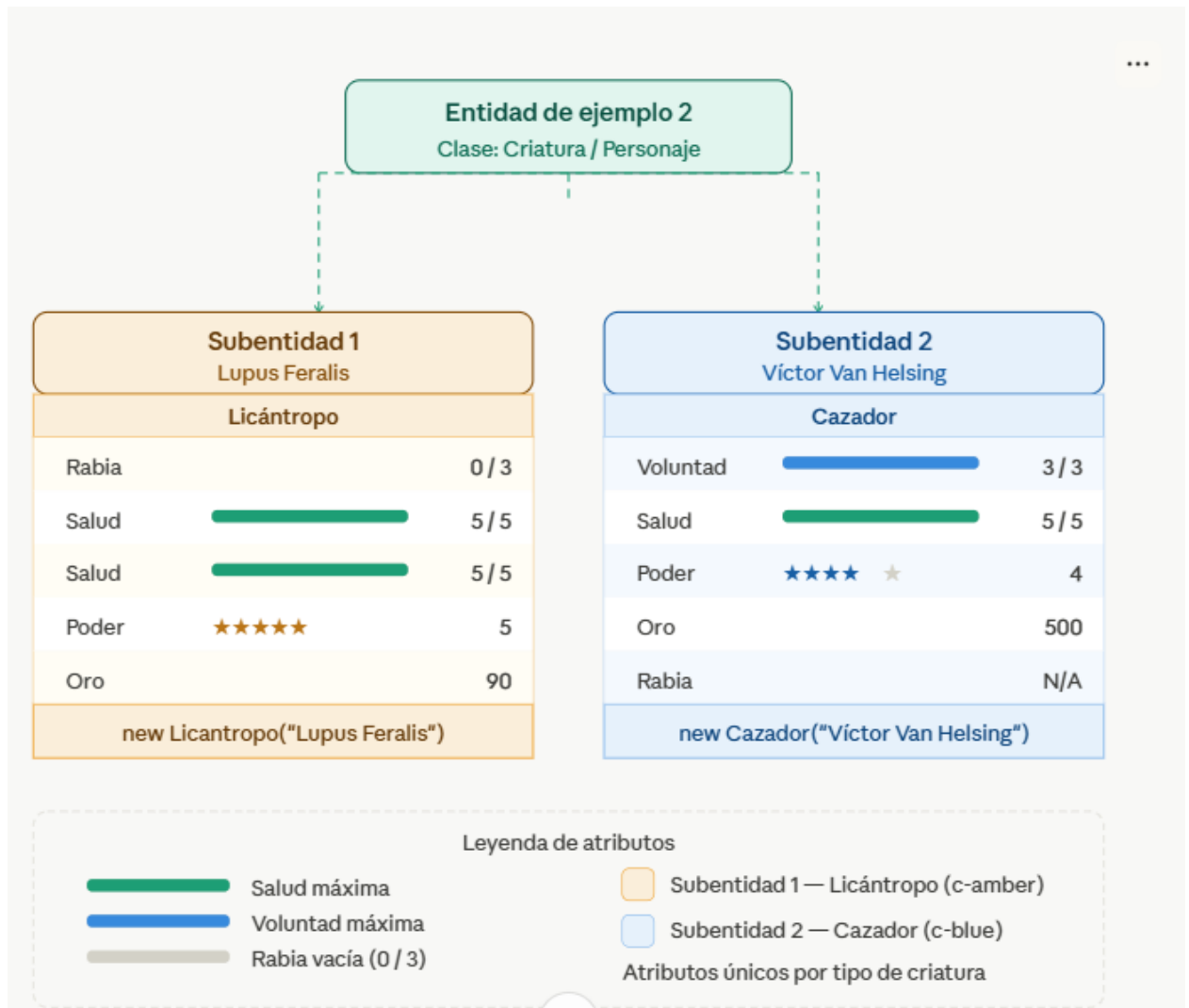


Ilustración 12 - Entidad ejemplo 2

Prototipos de pantallas

Los siguientes prototipos de pantalla ilustran la interfaz de consola de la aplicación. Representan los puntos de interacción más relevantes del flujo de usuario, incluyendo la bienvenida y el inicio de sesión.

```

┌────────────────────────────────────────────────────────────────────────────────┐
│                                     BIENVENIDO AL MUNDO DE LAS CRIATURAS                                     │
└────────────────────────────────────────────────────────────────────────────────┘

¿Qué deseas hacer?

[1] Iniciar sesión
[2] Registrarse como jugador
[3] Salir

> Introduce tu opción: _
```

Ilustración 13 Pantalla de bienvenida

La pantalla de inicio de sesión solicita al usuario su nick y contraseña. El sistema valida las credenciales y, si son correctas, redirige al menú correspondiente según el rol. Si el usuario tiene un desafío pendiente, se muestra una notificación y el acceso queda restringido hasta que lo resuelva.

```

Introduce tus credenciales:

Nick      : drakul_lord
Contraseña : *****

[ENTER] Confirmar  [0] Volver

> ¡Bienvenido de nuevo, drakul_lord!
> Tienes 1 notificación pendiente.
```

Ilustración 14 Pantalla de inicio de sesión